

MACROFIT AI

Resumen del segundo bloque de trabajo

DIA 2 | Backend, FastAPI y primera API funcional

Autor	Matias Gonzalez
Fecha	16 de julio de 2026
Rama de trabajo	feature/backend-setup
Repositorio	github.com/jootuns/macrofit-ai

Objetivo del documento

Dejar constancia clara y reutilizable del trabajo realizado durante el segundo bloque del proyecto: preparacion del backend, creacion de la primera API con FastAPI, validacion en navegador y registro de los cambios en Git y GitHub.

1. Resumen tecnico del dia

El segundo bloque se centro en dar el salto desde la planificacion hacia un backend funcional. Se preparo un entorno aislado de Python, se instalaron las dependencias necesarias, se creo la aplicacion FastAPI inicial y se valido que el servidor respondiera correctamente.

Herramientas y componentes utilizados

Elemento	Uso en el proyecto
Python 3.14.6	Lenguaje principal del backend.
Entorno virtual backend/.venv	Aislar dependencias del proyecto.
FastAPI 0.139.0	Framework para crear la API REST.
Uvicorn 0.51.0	Servidor ASGI para ejecutar la API.
Swagger / OpenAPI	Documentacion automatica de los endpoints.
Git + GitHub	Control de versiones y copia remota del codigo.

Principales aprendizajes

- El backend actua como intermediario entre el cliente y la base de datos.
- Una API REST expone recursos mediante HTTP y devuelve respuestas JSON.
- Un entorno virtual permite aislar dependencias de otros proyectos.
- FastAPI genera documentacion interactiva de forma automatica.
- Uvicorn es el servidor que ejecuta la aplicacion durante el desarrollo.
- La recarga automatica con --reload facilita iterar rapidamente durante la fase de desarrollo.

Arquitectura seguida en este bloque

Aplicacion movil -> API REST -> Backend -> Base de datos
En este momento solo se implemento la capa de backend basica. La aplicacion movil y la base de datos se seguiran desarrollando en fases posteriores.

2. Trabajo realizado

Se creo una rama especifica para el backend, se preparo el entorno virtual y se escribio el primer archivo funcional de FastAPI.

Rama y estructura

Elemento	Estado
Rama creada	feature/backend-setup
Carpeta principal del backend	backend/
Entorno virtual	backend/.venv/
Archivo de dependencias	backend/requirements.txt
Archivo principal de la API	backend/app/main.py

Dependencias instaladas en el entorno

- fastapi
- uvicorn
- pydantic
- starlette
- python-dotenv
- httpx
- rich
- watchfiles

Endpoints implementados

Metodo	Ruta	Descripcion
GET	/	Mensaje de bienvenida de la API.
GET	/health	Comprobacion rapida de estado del servidor.

Pruebas realizadas

- Acceso a http://127.0.0.1:8000/ con respuesta JSON correcta.
- Acceso a http://127.0.0.1:8000/health con respuesta {"status":"ok"}.
- Acceso a http://127.0.0.1:8000/docs para validar Swagger.
- Verificacion de que la recarga automatica estaba activa con --reload.

3. Git y GitHub

Todo el trabajo realizado en este bloque quedo guardado en un commit especifico y sincronizado con la rama remota correspondiente.

Commit y subida

Elemento	Valor
Commit	feat: initialize backend with fastapi
Hash	0d146d6
Rama local	feature/backend-setup
Rama remota	origin/feature/backend-setup
Estado final	working tree clean

Estado del proyecto al cerrar el dia

- La API responde correctamente en la raiz /.
- El endpoint /health devuelve un estado valido.
- Swagger documenta automaticamente los endpoints.
- El backend ya tiene una base funcional y versionada.
- La rama feature/backend-setup esta lista para seguir evolucionando o integrarse mas adelante.

Proximos pasos recomendados

- Separar el backend por capas: api, services, repositories, schemas y models.
- Crear una estructura mas limpia y profesional antes de añadir nuevas funcionalidades.
- Empezar con el primer endpoint de negocio para el calculo de macros.
- Actualizar el backlog y documentar el dia 2 en PDF como parte del historico del proyecto.